

無線操縦 (teleop) の調整手順

人の手で値を変えて、実機で確かめるまで

2026-08-29

- 0. この文書について
- 1. 全体像
- 2. 毎回の準備
- 3. 調整のしかた — 3 通り
- 4. 項目の一覧
- 5. よくある調整 — 手順つき
- 6. 変えてはいけない・気を付ける値
- 7. 困ったとき
- 8. 変更をリポジトリに残す
- 付録 A. コマンド早見表
- 付録 B. 差分ファイルの雛形
- 付録 C. この文書の PDF を作る

0. この文書について

PS5 コントローラで機体を動かす teleop ノード (`src/roboone_teleop`) の **値を人の手で変える** ための手順書。対象は「速さ・遊び・ボタン・長押し時間・音と光」のような調整項目で、歩き方そのもの (歩幅・周期・重心高さ) は `roboone_motion/config/gait.yaml` の担当なので別 (§ 5.1 に関係だけ書く)。

前提は Raspberry Pi 5 / ROS 2 Jazzy / ワークスペースは `~/ros2_ws`。コマンドは Pi の端末で打つ。この文書と同じ内容の要約が `docs/commands.md` の「teleop の調整」節にあり、コードの入口は `src/roboone_teleop/README.md`。

コードの側で何を変えたか (2026-08-29)

これまでも値は `config/ps5_dualsense.yaml` に集まっていたが、人が触るには 3 つの壁があった。それぞれをコード側で潰してある。

壁	直したこと
YAML を直しても <code>colcon build</code> し直さないと効かない	<code>--symlink-install</code> で入れ直す手順を § 3.2 に。以後は再起動だけで効く
走らせたまま <code>ros2 param set</code> しても、起動時に読んだ値が残って効かない	ノードが変更を検査して次の周期に読み直す。全項目が走らせたまま変えられる
キーを打ち間違えると ROS が黙って捨てる	起動時に「知らないキー」と警告（近い名前も出す）。 <code>teleop_params --check</code> で起動前にも検査できる
割り当てた技名が本当に動くか誰も確かめない	起動時に <code>motions.yaml</code> と照合して無い名前を警告。 <code>teleop_params --check --motions</code> で起動前にも

あわせて、項目の**一覧・意味・単位・既定値・範囲**を `roboone_teleop/params.py` の表 1 か所に集めた。コードの中に散っていた既定値は全部そこへ移してあり、**人が読むべきコードはその表だけ**でよい。

1. 全体像

1.1 部品の地図

ファイル	役割	人が触るか
<code>src/roboone_teleop/config/ps5_dualsense.yaml</code>	調整値の本体 。全項目がコメント付きで並ぶ	触る
<code>~/teleop_overrides.yaml</code> （任意・自分で作る）	上の YAML の上に重ねる差分。変えたいキーだけ書く	触る
<code>src/roboone_teleop/roboone_teleop/params.py</code>	項目の表（名前・意味・単位・既定・範囲）。検査と一覧の出力もここ	読む。項目を 増やす ときだけ触る
<code>src/roboone_teleop/roboone_teleop/teleop_node.py</code>	ノード本体。表を読んで宣言し、検査して反映する	触らない
<code>src/roboone_teleop/roboone_teleop/bindings.py</code>	"b10" / "a7-" のようなボタン表記の解釈	触らない
<code>src/roboone_teleop/launch/teleop.launch.py</code>	joy ノードと teleop を上げる。 <code>config:= overrides:=</code> を受ける	触らない
<code>src/roboone_bringup/launch/roboone.launch.py</code>	機体一式の launch。 <code>teleop_config:=</code> <code>teleop_overrides:=</code> を teleop に渡す	触らない
<code>src/roboone_teleop/test/test_params.py</code>	表と YAML の整合（打ち間違い・型・範囲）を見張る。ROS 不要	触らない
<code>src/roboone_teleop/test/test_teleop.py</code>	結線テスト（デッドマン・ウォッチ・ドッグ・走らせたままの調整）	触らない

1.2 値の効く順番

同じ項目が複数の場所にあるときは、**後のものが勝つ**。

- `params.py` の表の既定値 — `config` を渡さずに起動したときだけ使われる

2. `config/ps5_dualsense.yaml` — `launch` の既定。普段はこれが効いている
3. `overrides:=<自分の YAML>` — 2 の上に重ねる。書いたキーだけ上書き
4. `ros2 param set` — 走らせたまま上書き。ノードを再起動すると消える

「いま効いている値」は `ros2 param get /teleop <名前>` か、起動ログの 調整値: の行で分かる。

1.3 守り

- `config` に表にない名前があると、起動時に `config` の `"scal.x"` は `teleop` に無い項目なので効いていない (`"scale.x"` の打ち間違い?) と警告する。
- 型と範囲の外れた値は、起動時 (YAML) は例外で止まり、走行中 (`ros2 param set`) は拒否されて理由が返る。今の値は守られる。
- 数値は `1` でも `1.0` でもよい (int / float を区別しない)。`true` / `false` は引用符なし。
- 走行中の変更は「検査 → 次の周期に反映」なので、途中の値で動くことはない。
- `motion_bindings` の技名は起動時に `motions.yaml` と照合し、無い名前は 技 `"turn_1"` は `motions.yaml` に無い。割り当てたボタンを押しても何も起きない と警告する。

2. 毎回の準備

2.1 端末を開いたら source する

ROS のコマンド (`ros2 ...`) と、このワークスペースのパッケージは、端末ごとに読み込みが要る。新しい端末を開くたびに 2 行。

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
```

毎回打つのが面倒なら `~/.bashrc` の末尾に同じ 2 行を足しておく。 `colcon build` したあとは、その端末で `source ~/ros2_ws/install/setup.bash` を打ち直す (古い読み込みが残る)。

2.2 コントローラが繋がっているか

```
ls /dev/input/js0          # あれば OS には見えている (無ければ README §1 のペアリング)
ros2 topic hz /joy         # teleop.launch を上げた状態で 20Hz 前後なら OK
```

2.3 機体を動かさずに操縦系だけ試すとき

サーボにトルクを入れずに、指令が出るところまでを見る。

```
ros2 launch roboone_bringup roboone.launch.py allow_torque:=false
ros2 topic echo /cmd_walk      # R1 + 左スティックで Linear.x / Linear.y が動く
```

★ `allow_torque` を付けない既定はトルクが入る (機体が動く)。起動前に機体を支えておくこと。

3. 調整のしかた — 3 通り

方法	向いている場面	残るか
A. 走らせたまま試す	「0.06 と 0.08 でどう違うか」を実機で往復したい	再起動で消える
B. YAML を直して再起動	決まった値を残す。人に渡す	パッケージに残る
C. 自分用の差分ファイル	パッケージの YAML を汚さず、手元の値で回す	自分の home に残る

普段は **A** で探して、決まったら **B** に写す。

3.1 A. 走らせたまま試す

teleop が上がっている状態で、別の端末から (source を忘れずに)。

```
ros2 param list /teleop          # 項目の名前一覧
ros2 param describe /teleop scale.x  # 意味・単位・範囲 (表から出る)
ros2 param get /teleop scale.x      # いまの値
ros2 param set /teleop scale.x 0.08  # 変える。次の周期 (50ms 以内) から効く
```

- 効いたかは teleop の端末に 調整: scale.x = 0.08 (次の周期から効く) と 調整値: ... の 2 行が出ることで分かる。
- 拒否されたときは Setting parameter failed: scale.x: 0.5 以下にすること のように理由が返る。今の値は変わらない。
- リストはこう書く: ros2 param set /teleop motion_bindings "[b1:punch_r, b2:punch_l]"
- 文字列はそのまま: ros2 param set /teleop buttons.relax b9
- 決まった値をまとめて見るには ros2 param dump /teleop。出てくるのは YAML そのものなので、そこから B の YAML へ写せる。

3.2 B. YAML を直して再起動

編集するファイルは ~/ros2_ws/src/roboone_teleop/config/ps5_dualsense.yaml。

```
nano ~/ros2_ws/src/roboone_teleop/config/ps5_dualsense.yaml  # 好きなエディタで
ros2 run roboone_teleop teleop_params --check \
~/ros2_ws/src/roboone_teleop/config/ps5_dualsense.yaml      # 保存したら検査
```

検査が OK 問題なし なら、teleop を上げ直す (Ctrl-C して同じ launch)。起動ログの 調整値: 行で効いていることを見る。

編集が効かない (再ビルドが要る) 状態かどうか。colcon build を普通に回すと YAML は install/ へコピーされるので、src/ を直しても効かない。symlink で入れてあれば src/ の編集がそのまま効く。どちらか分からなければ:

```
ls -la ~/ros2_ws/install/roboone_teleop/share/roboone_teleop/config/
# ps5_dualsense.yaml -> /home/auto/ros2_ws/src/... のように矢印が出れば symlink (編集が効く)
# 矢印が無ければコピー (再ビルドが要る)
```

コピーになっていたら、**一度だけ** symlink で入れ直す。build と install を消すのは、コピーと symlink が混ざると古い方が残ることがあるため。

```
cd ~/ros2_ws
rm -rf build/roboone_teleop install/roboone_teleop
colcon build --packages-select roboone_teleop --symlink-install
source install/setup.bash
```

以後は YAML と Python の編集が再起動だけで効く。**ファイルを新しく足したとき** (config に別の YAML を増やす等) だけは もう一度 `colcon build --packages-select roboone_teleop --symlink-install`。

3.3 C. 自分用の差分ファイル

パッケージの YAML を触らずに、自分の値で回したいとき。変えたいキー**だけ**書く。

```
# ~/teleop_overrides.yaml
/**:
  ros__parameters:
    scale:
      x: 0.08
      y: 0.04
    deadzone: 0.15
```

```
ros2 run roboone_teleop teleop_params --check ~/teleop_overrides.yaml # 検査 (無い項目は「既定値が使われる」と出るだけ)
ros2 launch roboone_teleop teleop.launch.py overrides:=~/teleop_overrides.yaml
ros2 launch roboone_bringup roboone.launch.py teleop_overrides:=~/teleop_overrides.yaml
```

パッケージの YAML の上に重なるので、書かなかった項目はパッケージの値のまま。決まったら B に写して、差分ファイルは消す (2 か所に値があると、次に読む人がどちらが正か迷う)。

4. 項目の一覧

`ros2 run roboone_teleop teleop_params` で端末に出るものと同じ。**表を直したらこの節も差し替える** (`ros2 run roboone_teleop teleop_params --md` で Markdown が出る)。

ボタンの表記は `b<番号>` (ボタン) と `a<番号>+ / a<番号>-` (軸を ± 0.5 で閾値判定。十字キーが軸として出る機種向け)。番号は `ros2 run roboone_teleop joy_probe` で実機から読む (§ 5.3)。

周期・安全

名前	既定	単位	範囲	意味
rate_hz	20.0	Hz	5 ～ 100	/cmd_walk の送信周期。motion の cmd_timeout (0.5 s) より十分速く
joy_timeout	0.5	s	0.1 ～ 5	/joy がこれだけ途切れたら脱力をラッチ。大きくすると電波断に気付くのが遅れる
deadzone	0.12		0 ～ 0.9	スティックの遊び (0..1)。触っていないのに歩き出すなら上げる

スティック

名前	既定	単位	範囲	意味
axes.walk_x	1		0 ～ 31	前後 (linear.x) に使う軸番号。joy_probe で確かめる
axes.walk_y	0		0 ～ 31	左右の並行移動 (linear.y) に使う軸番号
axes.walk_yaw	2		0 ～ 31	旋回 (angular.z) に使う軸番号。今の motion は使わない
invert.walk_x	false			前後の符号を反転。joy ノードが ROS 規約へ反転済みなので普段は false
invert.walk_y	false			左右の符号を反転
invert.walk_yaw	false			旋回の符号を反転
scale.x	0.06	m/s	0 ～ 0.5	スティック全倒しの前後速度。walk_core の v_max (gait.yaml) で頭打ち
scale.y	0.03	m/s	0 ～ 0.5	スティック全倒しの左右速度。v_max の y で頭打ち
scale.yaw	0.4	rad/s	0 ～ 3	スティック全倒しの旋回速度
accel.x	0.15	m/s ²	0.01 ～ 10	前後指令の変化率の上限。gait.yaml の a_max より大きくしても motion 側で削られる
accel.y	0.1	m/s ²	0.01 ～ 10	左右指令の変化率の上限
accel.yaw	1.5	rad/s ²	0.01 ～ 30	旋回指令の変化率の上限

ボタン

名前	既定	単位	範囲	意味
buttons.deadman	"b10"			押している間だけ歩行・技が通る (R1)
buttons.relax	"b9"			押した瞬間に脱力 (L1)
buttons.home	"b6"			home_hold 秒の長押しでホームポジション → トルクオン (Options)
buttons.link_test	"b4"			押している間プザー + LED で電波の疎通を示す (Create)
buttons.autonomy	"b11"			autonomy_hold 秒の長押しで自律動作へ (十字キー 上)
buttons.hold	"b7"			hold_hold 秒の長押しで、今の姿勢のままトルクを入れる (L3)。転倒 → 脱力 → 起き上がりの経路用

ホームポジション / その場保持

名前	既定	単位	範囲	意味
home_hold	1.0	s	0 ～ 10	長押し時間。誤発動防止なので短くしない
home_motion	"home"			先に /cmd_motion へ送る技名
home_torque_delay	0.1	s	0 ～ 5	home / hold を送ってから /estop false (トルクオン) までの間
hold_hold	1.0	s	0 ～ 10	その場保持で武装する長押し時間
hold_motion	"hold"			その場保持で /cmd_motion へ送る名前。motion ノードが特別扱いする (motions.yaml には書かない)

自律動作

名前	既定	単位	範囲	意味
autonomy_hold	1.0	s	0 ～ 10	自律動作に入る長押し時間
autonomy.stop_on_joy_loss	true			電波が切れたら自律も止めて脱力する。false にすると非常停止の手が無くなる

技

名前	既定	単位	範囲	意味
motion_bindings	[b1:punch_r, b2:punch_l, b3:getup_front, b0:getup_back, b13:turn_l, b14:turn_r, b12:squat]			“<割り当て>:<技名>” の並び。技名は motions.yaml と一致させる
motion_interrupts	[getup_front, getup_back]			デッドマン不要で、押すと自律動作を止める技
motion_requires_deadman	true			上記以外の技はデッドマンを押している間だけ通す
motion_cooldown	0.5	s	0 ～ 5	同じ技を続けて送らない間隔
motions_yaml	""			技名の照合に使う motions.yaml の場所。空なら roboone_motion_node の share から探す

無線テスト

名前	既定	単位	範囲	意味
link_test.buzzer	"beep"			ui ノードのブザープリセット名 (beep / ack / error)
link_test.buzzer_hz	15.0	Hz	1 ～ 50	ブザーを撃ち直す周期
link_test.stale	0.15	s	0.02 ～ 2	/joy がこれだけ途切れたら即鳴り止む (joy_timeout を待たない)

状態表示 (ui ノードへ)

名前	既定	単位	範囲	意味
<code>ui.enable</code>	<code>true</code>			OLED / LED / ブザーへの状態表示を出すか
<code>ui.pattern.nolink</code>	<code>"dark"</code>			/joy 未受信の LED プリセット
<code>ui.pattern.relax</code>	<code>"estop"</code>			脱力中の LED プリセット
<code>ui.pattern.unarmed</code>	<code>"warn"</code>			再武装待ちの LED プリセット
<code>ui.pattern.manual</code>	<code>"ready"</code>			武装済み (歩ける) の LED プリセット
<code>ui.pattern.auto</code>	<code>"auto"</code>			自律動作中の LED プリセット
<code>ui.pattern.link</code>	<code>"link"</code>			無線テスト中の LED プリセット
<code>ui.buzzer.relax</code>	<code>"error"</code>			脱力に落ちたときの音
<code>ui.buzzer.auto</code>	<code>"ack"</code>			自律動作に入ったときの音
<code>ui.buzzer.manual</code>	<code>"beep"</code>			武装した (歩ける状態になった) ときの音

5. よくある調整 — 手順つき

5.1 歩く速さを変える (`scale.x` / `scale.y`)

スティックを全部倒したときの速度指令。歩幅は **motion 側で決まる**ので、ここを上げても `gait.yaml` の `v_max: [0.15, 0.08]` を超えた分は motion が切る。また指令の変化率は `accel.*` と `gait.yaml` の `a_max: [0.15, 0.05]` の小さい方で制限される。

- 機体を支えるか、`allow_torque:=false` で指令だけ見る
- `ros2 param set /teleop scale.x 0.08` (3 割ずつ上げる。いきなり倍にしない)
- R1 + 左スティック前倒しで歩かせ、`ros2 topic echo /cmd_walk` の `linear.x` が狙いの値になるのを見る
- 決まったら `config/ps5_dualsense.yaml` の `scale: x:` に写して再起動 (§ 3.2)

`scale.y` は横歩き。斜めに倒すと x と y が同時に立つので、合成速度は最大 $\sqrt{(x^2+y^2)}$ になる。

5.2 スティックの遊び (`deadzone`)

DualSense は中央が少しずれるので、触っていないのに `/cmd_walk` に小さい値が乗ることがある。

- R1 を押したままスティックから手を離し、`ros2 topic echo /cmd_walk` が `0.0` が見る
- 乗っているなら `ros2 param set /teleop deadzone 0.15` のように少しずつ上げる
- 上げすぎると倒し始めの反応が鈍る。不感帯の外は 0.1 に引き伸ばすので段差は出ない

joy ノード側の `deadzone` は launch で 0 に固定してある (二重に効くと読めなくなる)。触らない。

5.3 ボタンを差し替える (`buttons.*` / `motion_bindings`)

番号は推測しない。必ず実機で読む。


```
ros2 launch roboone_teleop teleop.launch.py          # 端末 1 (joy ノードが要る)
ros2 run  roboone_teleop joy_probe                  # 端末 2
```

押したボタンが `button 9` ↓ 押した `config` には `"b9"` (`L1?`) のように出る。出た文字列をそのまま YAML に書く。十字キーが `axis 7 = -1.00 ... "a7-"` のように軸で出る機種なら `"a7-"` と書く。

```
buttons:
  relax: "b9"
motion_bindings:
  - "b1:punch_r"
  - "a7-:squat"
```

走らせたままなら `ros2 param set /teleop buttons.relax b9`。書式が壊れていれば拒否される。

5.4 長押し時間とウォッチドッグ (`home_hold` / `autonomy_hold` / `joy_timeout`)

- `home_hold` と `autonomy_hold` は誤操作でトルクが入る・自律に入るのを防ぐための時間。短くするなら理由を持って。
- `joy_timeout` は「電波が切れてから脱力するまで」。0.5 s は Bluetooth の途切れ (数十 ms) では反応せず、切断 (数百 ms 以上) では確実に落ちる値。大きくすると転倒の前に止められなくなる。
- `link_test.stale` は無線テストのブザーが止まるまでの時間で、こちらは短いほど電波の切れ目が分かりやすい。

5.5 技を足す・ボタンに割り当てる

技の中身 (キーフレーム) は `src/roboone_motion_node/config/motions.yaml1`、ボタンへの割り当てが `teleop` の `motion_bindings`。技名は両方で一致させる。

1. `motions.yaml1` に技を書く (作り方は `docs/commands.md` の「モーションを作る」)
2. `motion_bindings` に `"b<番号>:<技名>"` を足す
3. 押して `/cmd_motion` にその名前が出るのを `ros2 topic echo /cmd_motion` で見る

`motion` 側に無い名前を押しても、`motion` が「知らない技」と出すだけで何も起きない。2026-08-29 時点で `turn_l` / `turn_r` は `teleop` に割り当てがあるが `motions.yaml1` に定義が無い (押しても何も起きない)。

起き上がりのように**転んだ機体を起こす技**は `motion_interrupts` に入れる。デッドマン不要になり、自律動作中でも通る。

5.6 LED と音を変える (`ui.pattern.*` / `ui.buzzer.*`)

`teleop` は色を持たない。ui ノードのプリセット名を送るだけなので、ここで選べるのはプリセット名。

種類	使える名前 (2026-08-29 時点)	定義場所
LED	dark ready warn estop auto link	src/roboone_ui/roboone_ui/ui_node.py の LED_PATTERNS
ブザー	beep ack error	同 BUZZER_PATTERNS

色そのものや点滅の速さを変えたいときは ui 側の表を直す (teleop は触らない)。無い名前を送ると ui が「未知のパターン、現状維持」と出して何もしない。

5.7 転倒から復帰する (その場保持で武装 → 起き上がり)

転倒すると電波の瞬断か L1 で脱力に入る。ここで Options (ホーム) を押すと立位へ 2 秒かけて動き出すので、寝ている機体では脚が床を押して跳ねる。代わりに:

1. **L3 を 1 秒長押し** → `/cmd_motion hold`、0.1 秒後に `/estop false`。motion は今の実測姿勢のまま トルクを入れる (`/motion/state` が `ARMING` → `STAY`)
2. △ (うつ伏せから) / × (仰向けから) で起き上がり。終わると `HOLD` (立位のスタンス) に戻る
3. 歩くには一度 R1 を離して再武装

関係する項目: `buttons.hold` (既定 L3 = b7、**実機未照合**)、`hold_hold` (長押し時間)、`hold_motion` (`hold`)、motion 側の `hold_arm_time` (`roboone_motion_node/config/motion_node.yaml`)。

★ 初回は `allow_torque:=false` で試すこと。 `STAY` に入ったら `/motion/joint_commands` と `/joint_states` を見比べ、寝た姿勢が IK を往復して同じ値に戻っているかを確認める (大きく違う軸があれば、トルクを入れた瞬間に そこが跳ねる)。それからトルクありに進む。

6. 変えてはいけない・気を付ける値

値	理由
launch の <code>autorepeat_rate</code> を 0 にする	スティックを動かさない限り <code>/joy</code> が来なくなり、静止と電波断の区別が付かなくなる
<code>invert.*</code> を true にする	joy ノードが既に ROS 規約へ反転済み。逆に動くときは先に README §5 の実測表を疑う
<code>autonomy.stop_on_joy_loss</code> を false にする	電波が切れても自律が続く。物理スイッチなど別の停止手段を用意してから
<code>joy_timeout</code> を大きくする	電波断からの脱力が遅れる
<code>rate_hz</code> を motion の <code>cmd_timeout</code> (0.5 s) の逆数に近づける	少し遅れただけで motion が「指令途絶」と見なす
<code>accel.*</code> を大きくする	速度指令が階段状に飛ぶ。二足はそれだけで転ぶ (motion 側の <code>a_max</code> でも削られるが、そちらに頼らない)

7. 困ったとき

症状	まず見るところ
YAML を直したのに変わらない	(1) <code>ros2 param get /teleop <名前></code> で今の値を確認 (2) symlink か (§ 3.2 の <code>ls -la</code>) (3) <code>source install/setup.bash</code> を打った端末か (4) <code>overrides:=</code> や <code>ros2 param set</code> が上書きしていないか (§ 1.2)
起動時に <code>config</code> の <code>"..."</code> は <code>teleop</code> に無い項目	キーの打ち間違い。警告に近い名前が出る。 <code>teleop_params --check</code> で起動前に拾える
起動時に例外で止まる (<code>InvalidParameterTypeException</code> など)	<code>true</code> / <code>false</code> を <code>"true"</code> と引用符付きで書いた、文字列に数値を書いた等。 <code>teleop_params --check</code> が同じことを日本語で言う
<code>ros2 param set</code> が <code>Setting parameter failed</code>	後ろの理由を読む (範囲・書式)。今の値は守られている
<code>/joy</code> が来ない (<code>ros2 topic hz /joy</code> が出ない)	コントローラのペアリング・電池。README § 1 と <code>docs/commands.md</code> 「実機まわり」
動いてほしい向きと逆	<code>invert.*</code> を触る前に README § 5 の実測表 (前 = <code>axes[1]</code> が +) と <code>joy_probe</code> で軸を確認
起動直後に R1 を押しても歩かない	再武装待ち。一度 R1 を離す (OLED に <code>release R1</code>)
技のボタンを押しても何も起きない	起動ログに 技 <code>"..."</code> は <code>motions.yaml</code> に無い が出ていないか。2026-08-29 時点では <code>turn_l</code> / <code>turn_r</code> がこれ。 <code>motions.yaml</code> に作るか割り当てを外す
転んだあと Options を押したら脚が跳ねた	ホームは立位へ動き出す。寝ている機体は L3 長押し (その場保持) → △/× (§ 5.7)
<code>ros2 run roboone_teleop teleop_params</code> が無いと言われる	<code>colcon build --packages-select roboone_teleop</code> のあと <code>source install/setup.bash</code>

8. 変更をリポジトリに残す

`ps5_dualsense.yaml` の変更は他の人にも効くので、決まった値は `git` に残す。差分ファイル (`~/teleop_overrides.yaml`) は残さない。

```
cd ~/ros2_ws
python3 -m pytest src/roboone_teleop/test/test_params.py -q      # 表と YAML の整合 (ROS 不要、1 秒)
colcon test --packages-select roboone_teleop && colcon test-result --verbose
git diff src/roboone_teleop/config/ps5_dualsense.yaml
git add src/roboone_teleop/config/ps5_dualsense.yaml
git commit -m "teleop: scale.x を 0.06 -> 0.08 (実機で歩かせて決めた)"
```

コミットメッセージには**なぜその値にしたか**を 1 行残す (「速くした」ではなく「何を見てそう決めたか」)。

ROS の無い PC で確かめる

Pi に繋げないときは `tools/` の道具で、ament と同じ規約の lint と結線テストを手元で回せる (`docs/commands.md` の「ROS の無い開発 PC で確かめる」、`tools/README.md`)。最終判定は Pi の `colcon test`。

項目を増やしたいとき

1. `params.py` の `TUNABLES` に 1 行足す (名前・既定・意味・見出し・単位・種類・範囲)
2. `teleop_node.py` の `_apply()` で `cfg['<名前>']` を読んで使う
3. `config/ps5_dualsense.yaml` にも同じキーを足す (`test_params.py` が「全項目が YAML にあること」を見張る)
4. `ros2 run roboone_teleop teleop_params --md` で §4 の表を出し直してこの文書に貼る

付録 A. コマンド早見表

準備 (端末ごと)

```
source /opt/ros/jazzy/setup.bash && source ~/ros2_ws/install/setup.bash
```

一覧と検査

```
ros2 run roboone_teleop teleop_params # 項目一覧 (意味・単位・既定・範囲)
ros2 run roboone_teleop teleop_params --check <yaml> # 編集した YAML の検査
ros2 run roboone_teleop teleop_params --check <yaml> --motions <motions.yaml> # 技名の照合も
ros2 run roboone_teleop joy_probe # ボタン・軸の番号を実機で読む
```

走らせたまま

```
ros2 param list /teleop
ros2 param describe /teleop scale.x
ros2 param get /teleop scale.x
ros2 param set /teleop scale.x 0.08
ros2 param dump /teleop
```

起動

```
ros2 launch roboone_teleop teleop.launch.py # teleop だけ
ros2 launch roboone_teleop teleop.launch.py overrides:=~/teleop_overrides.yaml
ros2 launch roboone_bringup roboone.launch.py allow_torque:=false # 機体を動かさず一式
ros2 launch roboone_bringup roboone.launch.py teleop_overrides:=~/teleop_overrides.yaml
```

見る

```
ros2 topic hz /joy
ros2 topic echo /cmd_walk
ros2 topic echo /cmd_motion
ros2 topic echo /estop
```

YAML の編集を再ビルド無しで効かせる (初回だけ)

```
rm -rf build/roboone_teleop install/roboone_teleop
colcon build --packages-select roboone_teleop --symlink-install && source install/setup.bash
```

テスト

```
python3 -m pytest src/roboone_teleop/test/test_params.py -q
# ROS の無い PC で (tools/README.md)
python tools/lint_like_ament.py
python tools/run_teleop_tests_without_ros.py
colcon test --packages-select roboone_teleop && colcon test-result --verbose
```

付録 B. 差分ファイルの雛形

`~/teleop_overrides.yaml` として保存し、`overrides:=~/teleop_overrides.yaml` で渡す。書いたキーだけが上書きされる。

```
/**:
  ros__parameters:
    # 速さ (スティック全倒し)
    scale:
      x: 0.08          # m/s
      y: 0.04          # m/s
    # スティックの遊び
    deadzone: 0.15
    # ボタン (joy_probe で読んだ番号)
    buttons:
      relax: "b9"
```

付録 C. この文書の PDF を作る

Windows の開発 PC で (pandoc と Edge が必要)。Pi 上では使わない。

```
powershell -ExecutionPolicy Bypass -File docs/build_md_pdf.ps1 docs/teleop_tuning.md
```

`docs/teleop_tuning.pdf` が同じ場所に出る。